

Database System Engineering and Distributed Backend Development

BANK DATABASE — SQL VIEWS

Name: Hansika

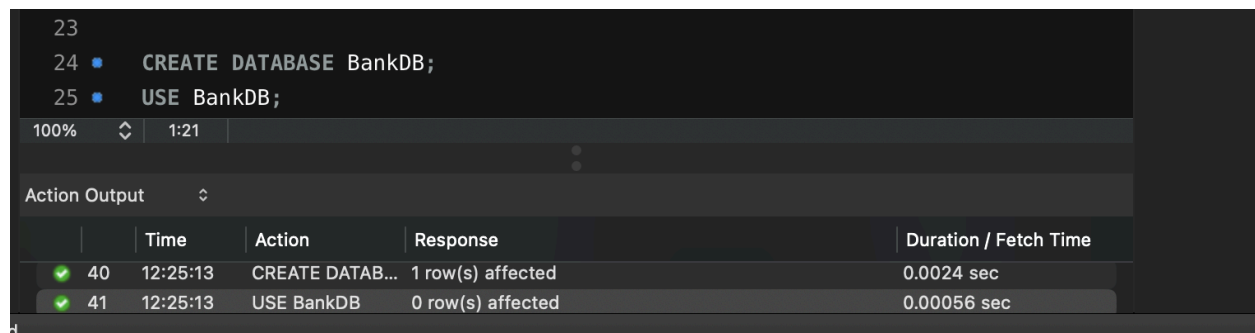
Roll No: 2520030237

Section: 7

Email: gunapatihansikareddy@klh.edu.in

QUERY 1 — Create Database

```
CREATE DATABASE BankDB;  
USE BankDB;
```



23						
24	•		CREATE DATABASE BankDB;			
25	•		USE BankDB;			
100%	↕	1:21				
Action Output ↕						
		Time	Action	Response		Duration / Fetch Time
✓	40	12:25:13	CREATE DATAB...	1 row(s) affected		0.0024 sec
✓	41	12:25:13	USE BankDB	0 row(s) affected		0.00056 sec

QUERY 2 — Create Customer Table

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15),  
    Email VARCHAR(100),  
    City VARCHAR(50)  
);
```

```
17
18 • CREATE TABLE Customer (
19     Customer_ID INT PRIMARY KEY,
20     Customer_Name VARCHAR(100) NOT NULL,
21     Phone VARCHAR(15),
22     Email VARCHAR(100),
23     City VARCHAR(50)
24 );
25
```

100% 1:17

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 42	12:25:53	CREATE TABLE...	0 row(s) affected	0.0097 sec

QUERY 3 — Create Account Table

```
CREATE TABLE Account (
    Account_No INT PRIMARY KEY,
    Customer_ID INT,
    Account_Type VARCHAR(20),
    Balance DECIMAL(12,2),
    Branch VARCHAR(50),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);
```

```
6
7
8 • CREATE TABLE Account (
9     Account_No INT PRIMARY KEY,
10    Customer_ID INT,
11    Account_Type VARCHAR(20),
12    Balance DECIMAL(12,2),
13    Branch VARCHAR(50),
14    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
15 );
16
```

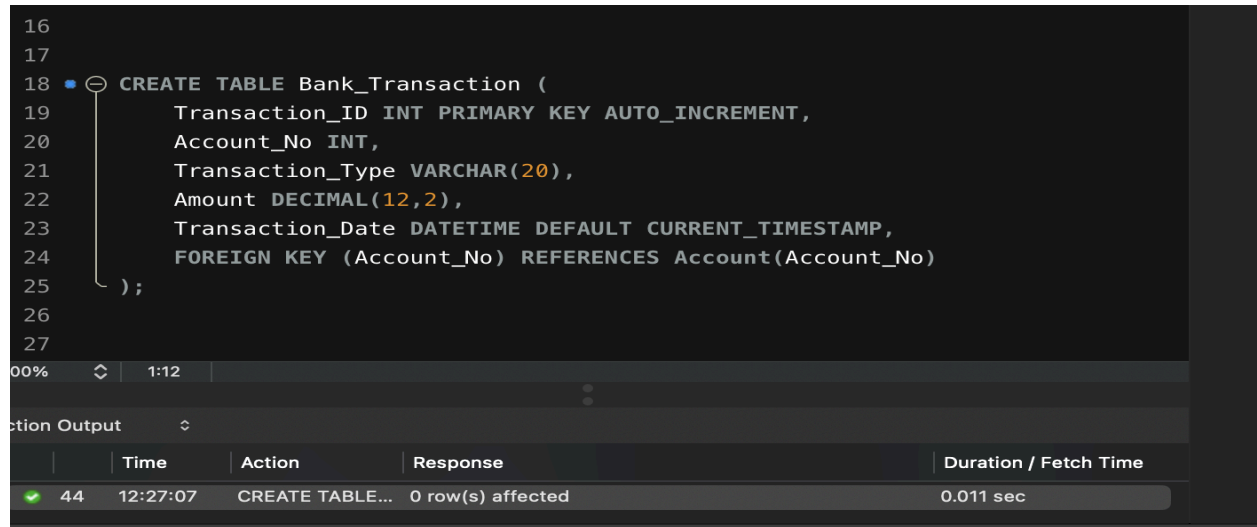
% 1:7

n Output

	Time	Action	Response	Duration / Fetch Time
43	12:26:37	CREATE TABLE...	0 row(s) affected	0.014 sec

QUERY 4 — Create Bank_Transaction Table

```
CREATE TABLE Bank_Transaction (  
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Account_No INT,  
    Transaction_Type VARCHAR(20),  
    Amount DECIMAL(12,2),  
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (Account_No) REFERENCES Account(Account_No)  
);
```



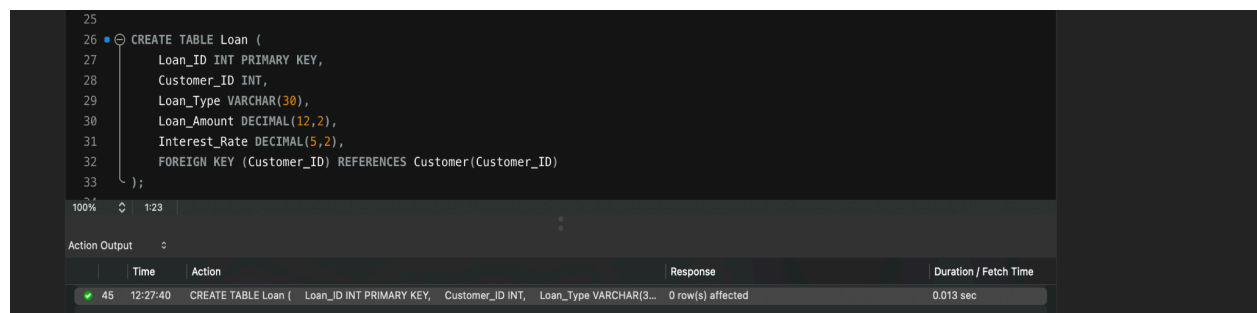
The screenshot shows a SQL IDE with a dark theme. The query editor displays the SQL code for creating the Bank_Transaction table. The query is executed, and the results are shown in the 'Action Output' pane at the bottom. The output table has columns: Time, Action, Response, and Duration / Fetch Time. The response indicates that 0 row(s) were affected and the duration was 0.011 sec.

```
16  
17  
18 CREATE TABLE Bank_Transaction (  
19     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
20     Account_No INT,  
21     Transaction_Type VARCHAR(20),  
22     Amount DECIMAL(12,2),  
23     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
24     FOREIGN KEY (Account_No) REFERENCES Account(Account_No)  
25 );  
26  
27
```

	Time	Action	Response	Duration / Fetch Time
✓ 44	12:27:07	CREATE TABLE...	0 row(s) affected	0.011 sec

QUERY 5 — Create Loan Table

```
CREATE TABLE Loan (  
    Loan_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Loan_Type VARCHAR(30),  
    Loan_Amount DECIMAL(12,2),  
    Interest_Rate DECIMAL(5,2),  
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
);
```



The screenshot shows a SQL IDE with a dark theme. The query editor displays the SQL code for creating the Loan table. The query is executed, and the results are shown in the 'Action Output' pane at the bottom. The output table has columns: Time, Action, Response, and Duration / Fetch Time. The response indicates that 0 row(s) were affected and the duration was 0.013 sec.

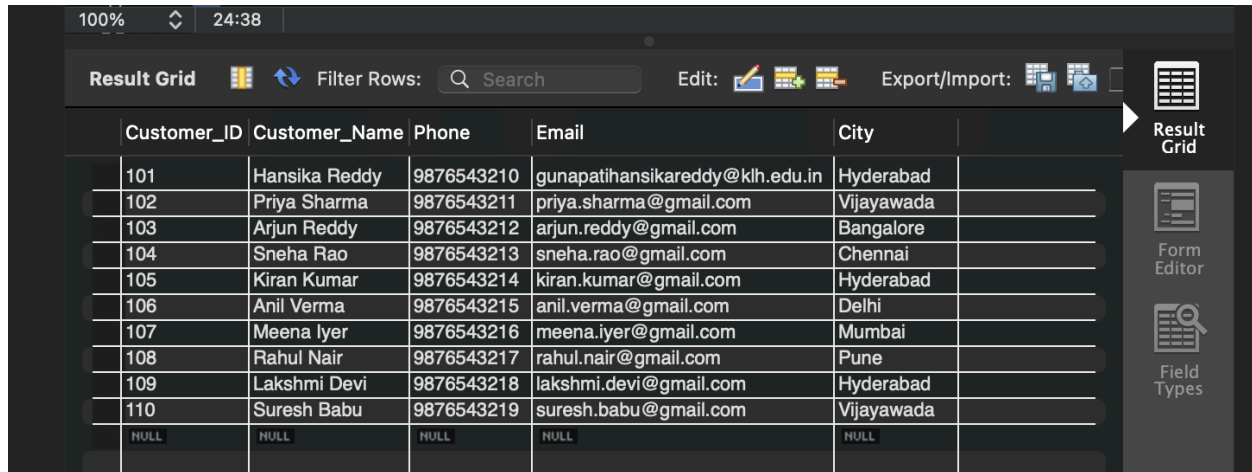
```
25  
26 CREATE TABLE Loan (  
27     Loan_ID INT PRIMARY KEY,  
28     Customer_ID INT,  
29     Loan_Type VARCHAR(30),  
30     Loan_Amount DECIMAL(12,2),  
31     Interest_Rate DECIMAL(5,2),  
32     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
33 );
```

	Time	Action	Response	Duration / Fetch Time
✓ 45	12:27:40	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30...	0 row(s) affected	0.013 sec

QUERY 6 — Insert Customer Records + Display

```
INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES
(101, 'Hansika Reddy', '9876543210', 'gunapatihansikareddy@klh.edu.in',
'Hyderabad'),
(102, 'Priya Sharma', '9876543211', 'priya.sharma@gmail.com', 'Vijayawada'),
(103, 'Arjun Reddy', '9876543212', 'arjun.reddy@gmail.com', 'Bangalore'),
(104, 'Sneha Rao', '9876543213', 'sneha.rao@gmail.com', 'Chennai'),
(105, 'Kiran Kumar', '9876543214', 'kiran.kumar@gmail.com', 'Hyderabad'),
(106, 'Anil Verma', '9876543215', 'anil.verma@gmail.com', 'Delhi'),
(107, 'Meena Iyer', '9876543216', 'meena.iyer@gmail.com', 'Mumbai'),
(108, 'Rahul Nair', '9876543217', 'rahul.nair@gmail.com', 'Pune'),
(109, 'Lakshmi Devi', '9876543218', 'lakshmi.devi@gmail.com', 'Hyderabad'),
(110, 'Suresh Babu', '9876543219', 'suresh.babu@gmail.com', 'Vijayawada');

SELECT * FROM Customer;
```



Customer_ID	Customer_Name	Phone	Email	City
101	Hansika Reddy	9876543210	gunapatihansikareddy@klh.edu.in	Hyderabad
102	Priya Sharma	9876543211	priya.sharma@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun.reddy@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha.rao@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran.kumar@gmail.com	Hyderabad
106	Anil Verma	9876543215	anil.verma@gmail.com	Delhi
107	Meena Iyer	9876543216	meena.iyer@gmail.com	Mumbai
108	Rahul Nair	9876543217	rahul.nair@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi.devi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh.babu@gmail.com	Vijayawada
NULL	NULL	NULL	NULL	NULL

QUERY 7 — Insert Account Records + Display

```
INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch)
VALUES
(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Hyderabad'),
(10006, 106, 'Savings', 65000, 'Delhi'),
(10007, 107, 'Current', 150000, 'Mumbai'),
(10008, 108, 'Savings', 35000, 'Pune'),
(10009, 109, 'Savings', 85000, 'Hyderabad'),
(10010, 110, 'Current', 110000, 'Vijayawada');

SELECT * FROM Account;
```

Account_No	Customer_ID	Account_Type	Balance	Branch	
10001	101	Savings	50000.00	Hyderabad	
10002	102	Savings	75000.00	Vijayawada	
10003	103	Current	120000.00	Bangalore	
10004	104	Savings	45000.00	Chennai	
10005	105	Current	90000.00	Hyderabad	
10006	106	Savings	65000.00	Delhi	
10007	107	Current	150000.00	Mumbai	
10008	108	Savings	35000.00	Pune	
10009	109	Savings	85000.00	Hyderabad	
10010	110	Current	110000.00	Vijayawada	
NULL	NULL	NULL	NULL	NULL	

QUERY 8 — Insert Bank Transaction Records + Display

```

INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES
(10001, 'DEPOSIT', 10000),
(10001, 'WITHDRAW', 5000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000),
(10006, 'DEPOSIT', 12000),
(10007, 'DEPOSIT', 25000),
(10008, 'WITHDRAW', 5000),
(10009, 'DEPOSIT', 20000),
(10010, 'WITHDRAW', 15000);

SELECT * FROM Bank_Transaction;

```

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date	
1	10001	DEPOSIT	10000.00	2026-08-22 12:30:17	
2	10001	WITHDRAW	5000.00	2026-08-22 12:30:17	
3	10002	DEPOSIT	15000.00	2026-08-22 12:30:17	
4	10003	WITHDRAW	20000.00	2026-08-22 12:30:17	
5	10004	DEPOSIT	5000.00	2026-08-22 12:30:17	
6	10005	WITHDRAW	10000.00	2026-08-22 12:30:17	
7	10006	DEPOSIT	12000.00	2026-08-22 12:30:17	
8	10007	DEPOSIT	25000.00	2026-08-22 12:30:17	
9	10008	WITHDRAW	5000.00	2026-08-22 12:30:17	
10	10009	DEPOSIT	20000.00	2026-08-22 12:30:17	
11	10010	WITHDRAW	15000.00	2026-08-22 12:30:17	
NULL	NULL	NULL	NULL	NULL	

QUERY 9 — Insert Loan Records + Display

```
INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),
(502, 102, 'Education Loan', 1000000, 6.5),
(503, 103, 'Car Loan', 800000, 8.2),
(504, 104, 'Personal Loan', 500000, 10.5),
(505, 105, 'Home Loan', 4000000, 7.2),
(506, 106, 'Car Loan', 900000, 8.5),
(507, 107, 'Business Loan', 3000000, 9.0),
(508, 109, 'Personal Loan', 600000, 10.0);

SELECT * FROM Loan;
```

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	
501	101	Home Loan	5000000.00	7.50	
502	102	Education Loan	1000000.00	6.50	
503	103	Car Loan	800000.00	8.20	
504	104	Personal Loan	500000.00	10.50	
505	105	Home Loan	4000000.00	7.20	
506	106	Car Loan	900000.00	8.50	
507	107	Business Loan	3000000.00	9.00	
508	109	Personal Loan	600000.00	10.00	
NULL	NULL	NULL	NULL	NULL	


Result Grid


Form Editor


Field Types

QUERY 10 — Simple View (View of an Entire Table)

```
CREATE VIEW Customer_View AS
SELECT * FROM Customer;

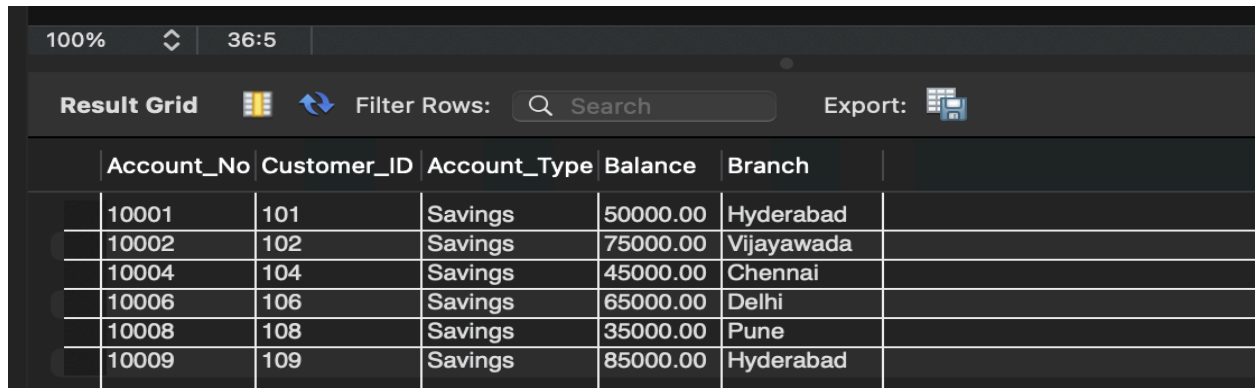
SELECT * FROM Customer_View;
```

Customer_ID	Customer_Name	Phone	Email	City	
101	Hansika Reddy	9876543210	gunapatihansikareddy@klh.edu.in	Hyderabad	
102	Priya Sharma	9876543211	priya.sharma@gmail.com	Vijayawada	
103	Arjun Reddy	9876543212	arjun.reddy@gmail.com	Bangalore	
104	Sneha Rao	9876543213	sneha.rao@gmail.com	Chennai	
105	Kiran Kumar	9876543214	kiran.kumar@gmail.com	Hyderabad	
106	Anil Verma	9876543215	anil.verma@gmail.com	Delhi	
107	Meena Iyer	9876543216	meena.iyer@gmail.com	Mumbai	
108	Rahul Nair	9876543217	rahul.nair@gmail.com	Pune	
109	Lakshmi Devi	9876543218	lakshmi.devi@gmail.com	Hyderabad	
110	Suresh Babu	9876543219	suresh.babu@gmail.com	Vijayawada	

QUERY 11 — View Using WHERE (Filtered View)

```
CREATE VIEW Savings_Account_View AS
SELECT * FROM Account
WHERE Account_Type = 'Savings';

SELECT * FROM Savings_Account_View;
```



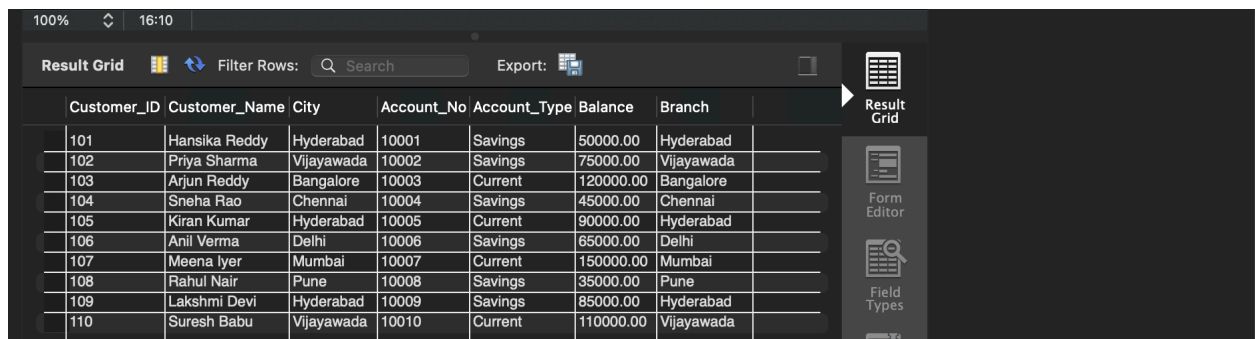
The screenshot shows a database application interface with a top bar displaying '100%' zoom and '36:5' time. Below the bar is a toolbar with 'Result Grid', 'Filter Rows', a search input, and an 'Export' button. The main area displays a table with the following data:

	Account_No	Customer_ID	Account_Type	Balance	Branch	
	10001	101	Savings	50000.00	Hyderabad	
	10002	102	Savings	75000.00	Vijayawada	
	10004	104	Savings	45000.00	Chennai	
	10006	106	Savings	65000.00	Delhi	
	10008	108	Savings	35000.00	Pune	
	10009	109	Savings	85000.00	Hyderabad	

QUERY 12 — View Using JOIN (Combining Two Tables)

```
CREATE VIEW Customer_Account_View AS
SELECT
    C.Customer_ID,
    C.Customer_Name,
    C.City,
    A.Account_No,
    A.Account_Type,
    A.Balance,
    A.Branch
FROM Customer C
JOIN Account A ON C.Customer_ID = A.Customer_ID;

SELECT * FROM Customer_Account_View;
```



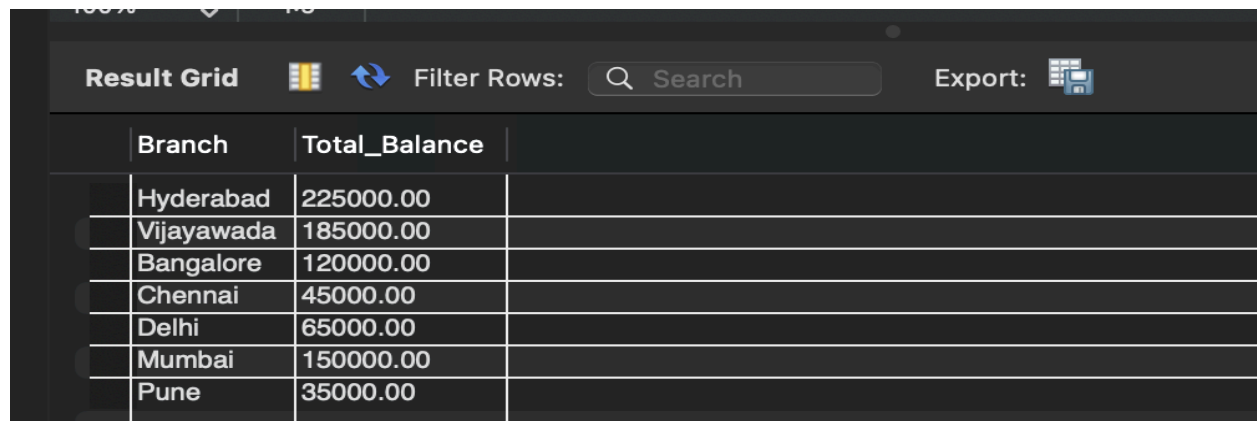
The screenshot shows a database application interface with a top bar displaying '100%' zoom and '16:10' time. Below the bar is a toolbar with 'Result Grid', 'Filter Rows', a search input, and an 'Export' button. The main area displays a table with the following data:

	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch	
	101	Hansika Reddy	Hyderabad	10001	Savings	50000.00	Hyderabad	
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada	
	103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore	
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai	
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad	
	106	Anil Verma	Delhi	10006	Savings	65000.00	Delhi	
	107	Meena Iyer	Mumbai	10007	Current	150000.00	Mumbai	
	108	Rahul Nair	Pune	10008	Savings	35000.00	Pune	
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad	
	110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada	

QUERY 13 — View Using GROUP BY + Aggregate Function

```
CREATE VIEW Branch_Total_Balance AS
SELECT
    Branch,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Branch;

SELECT * FROM Branch_Total_Balance;
```

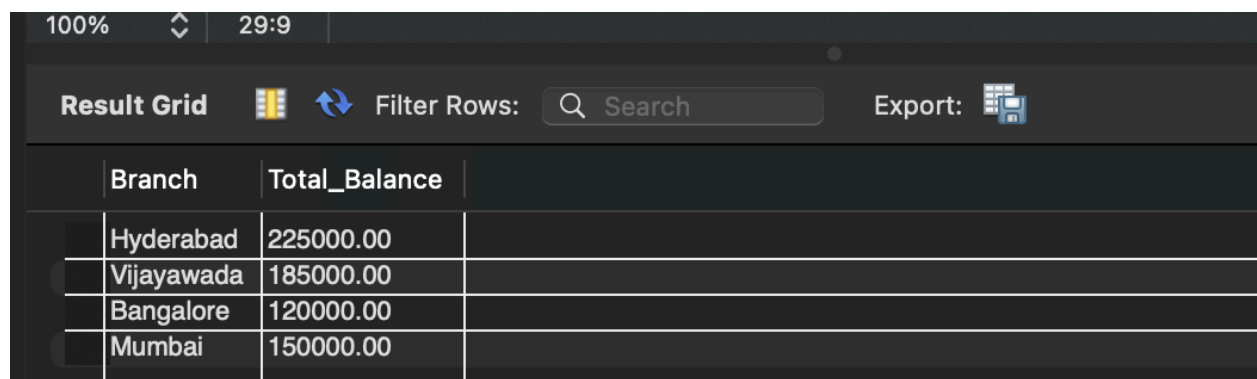


Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Chennai	45000.00
Delhi	65000.00
Mumbai	150000.00
Pune	35000.00

QUERY 14 — View Using HAVING (Filter After Grouping)

```
CREATE VIEW Rich_Branches AS
SELECT
    Branch,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Branch
HAVING SUM(Balance) > 100000;

SELECT * FROM Rich_Branches;
```



Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Mumbai	150000.00

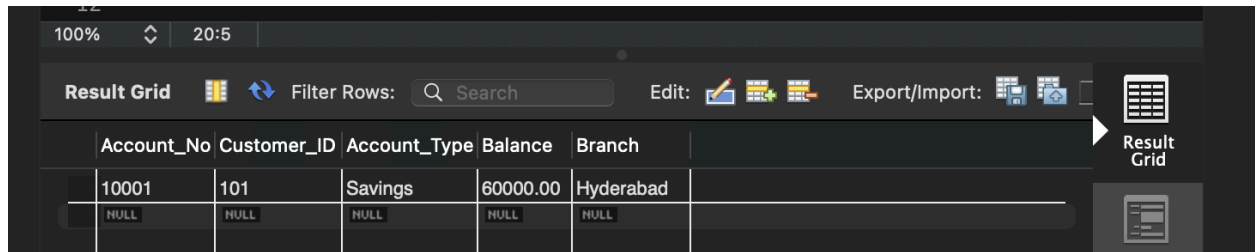
QUERY 15 — Update Through a View + Drop a View

```
CREATE VIEW Account_View AS
SELECT Account_No, Customer_ID, Account_Type, Balance, Branch
FROM Account;

UPDATE Account_View
SET Balance = 60000
WHERE Account_No = 10001;

SELECT * FROM Account WHERE Account_No = 10001;

DROP VIEW Account_View;
```



The screenshot shows a database application interface with a dark theme. At the top, there's a toolbar with icons for zooming (100%), a refresh icon, a clock (20:5), and a 'Result Grid' button. Below the toolbar is a search bar labeled 'Filter Rows:' and 'Search'. To the right of the search bar are icons for 'Edit' and 'Export/Import'. The main area displays a table with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

On the right side of the table, there's a vertical toolbar with a 'Result Grid' button and a 'Table' icon.